

Exercise 2: E-commerce Platform Search Function

Code:

```
//Requirements:
//Product class - Attributes allowing search operations.
//Search Interface - Single contract for both Searching algorithms.
//Binary Search class - Containing Binary Search functionality.
//Linear Search class - Consisting Linear Search Functionality.
//Sorting class - Implementing Merge Sort Algorithm since there is
possibility of arrays already being sorted that is why we prefer merge
sort over quick Sort.

import java.util.*;

public class EcommercePlatform{
    public static void main(String[] args){
        //Product Initiation
        Product p1 = new Product();
        Product p2 = new Product();
        Product p3 = new Product();
        Product p4 = new Product();

        p1.setProductId(40);
        p2.setProductId(20);
        p3.setProductId(10);
        p4.setProductId(30);

        p1.setProductName("Keyboard");
        p2.setProductName("Mouse");
        p3.setProductName("Moniter");
        p4.setProductName("GPU");

        p1.setCategory("tech-Peripheral");
        p2.setCategory("tech-Peripheral");
        p3.setCategory("tech-Essential");
        p4.setCategory("tech-Essential");

        List<Product> products = new ArrayList<>();
    }
}
```

```

        products.add(p1);
        products.add(p2);
        products.add(p3);
        products.add(p4);

        MergeSort.sort(products);

        for(Product p : products){
            System.out.println(p.getProductid() + " ");
        }
        System.out.println("-----Array
Sorted-----");
        Search ls = new LinearSearch();
        Product lp= ls.searchProduct(products,10);

        if(lp == null){
            System.out.println("Product Not Found");
        }else{
            System.out.println("Product Name : "+lp.getProductName());
            System.out.println("Product Category : "+lp.getCategory());
        }

        Search bs = new BinarySearch();
        Product bp = bs.searchProduct(products, 100);
        if(bp == null){
            System.out.println("Product Not Found");
        }else{
            System.out.println("Product Name : "+bp.getProductName());
            System.out.println("Product Category : "+bp.getCategory());
        }
    }
}

class Product{
    private int productId ;
    private String category ;
    private String productName;

    public void setProductId(int product_id){
        this.productId=product_id;
    }
}

```

```

    }

    public int getProductId(){
        return this.productId;
    }

    public void setCategory(String category){
        this.category = category;
    }

    public String getCategory(){
        return this.category;
    }

    public void setProductName(String product_name){
        this.productName = product_name;
    }

    public String getProductName(){
        return this.productName;
    }
}

interface Search{
    Product searchProduct(List<Product> products,int product_id);
}

class LinearSearch implements Search{

    public Product searchProduct(List<Product> products,int product_id){
        for(Product p: products){
            if(p.getProductId() == product_id){
                return p;
            }
        }
        return null;
    }
}

```

```

class BinarySearch implements Search{
    public Product searchProduct(List<Product> products,int product_id){
        int s=0;
        int e = products.size()-1;
        int mid = s+(e-s)/2;
        while(e>=s){
            if(product_id==products.get(mid).getProductId()){
                return products.get(mid);
            }
            else if(product_id<products.get(mid).getProductId()){
                e=mid-1;
                mid = s + (e-s)/2;
            }else{
                s=mid+1;
                mid = s+ (e-s)/2;
            }
        }

        return null;

    }

    public String getName(List<Product> products,int product_id){
        int s=0;
        int e = products.size()-1;
        int mid = s+(e-s)/2;
        while(e>=s){
            if(product_id==products.get(mid).getProductId()){
                return products.get(mid).getProductName();
            }
            else if(product_id<products.get(mid).getProductId()){
                e=mid-1;
                mid = s + (e-s)/2;
            }else{
                s=mid+1;
                mid = s+ (e-s)/2;
            }
        }

        return "Product Not Found";
    }
}

```

```
}
```

```
class MergeSort{  
    public static void sort(List<Product> products){  
        int s = 0;  
        int e = products.size() - 1;  
  
        divide(products,s,e);  
    }  
  
    private static void divide(List<Product> products, int s, int e){  
        if(e<=s){  
            return;  
        }  
        int mid = s+(e-s)/2;  
  
        divide(products, s, mid);  
        divide(products, mid+1, e);  
  
        merge(products, s, mid, e);  
    }  
  
    private static void merge(List<Product> products , int s, int mid, int  
e){  
        int i = s;  
        int j = mid+1;  
  
        List<Product> merged = new ArrayList<>();  
  
        while(i<=mid && j<=e){  
            if(products.get(i).getProductId() <=  
products.get(j).getProductId()){  
                merged.add(products.get(i++));  
            }else{  
                merged.add(products.get(j++));  
            }  
        }  
    }  
}
```

```
        while(i<=mid) {
            merged.add(products.get(i++));
        }
        while(j<=e) {
            merged.add(products.get(j++));
        }

        int k=s;
        for(Product p : merged){
            products.set(k++,p);
        }
    }
}
```

Output:

J EcommercePlatform.java X

Module-2 Data Structures and Algorithm > J EcommercePlatform.java > EcommercePlatform

```
1 //Requirements:
2 //Product class - Attributes allowing search operations.
3 //Search Interface - Single contract for both Searching algorithms.
4 //Binary Search class - Containing Binary Search functionality.
5 //Linear Search class - Consisting Linear Search Functionality.
6 //Sorting class - Implementing Merge Sort Algorithm since there is possibility of a
7
8 import java.util.*;
9
10 public class EcommercePlatform{
11     //Product Initiation
12     Product p1 = new Product();
13     Product p2 = new Product();
14     Product p3 = new Product();
15     Product p4 = new Product();
16
17     p1.setProductId(product_id: 40);
18     p2.setProductId(product_id: 20);
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
illing\Coding'; & 'C:\Program Files\Java\jdk-25.0.2\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\adits\AppData\Roaming\Code\User\workspaceStorage\3385de95c309e825d680cb68353935b\redhat.java\jdt_ws\Coding_613b24a0\bin' 'EcommercePlatform'
10
20
30
40
-----Array Sorted-----
Product Name : Moniter
Product Category : tech-Essential
Product Not Found
PS C:\Users\adits\Desktop\DeepSkilling\Coding>
```

powerShell

powerShell

Run: Ecomm...